

1. Effizienz von Algorithmen

1.1. Asymptotische Notation

$g = O(f)$	g wächst maximal so schnell wie f $0 \leq \lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} < \infty$ $O(f) := \{g : \mathbb{R} \rightarrow \mathbb{R} \mid \exists c > 0, n_0 \in \mathbb{N} : \forall n > n_0 : g(n) \leq cf(n)\}$
$g = \Omega(f)$	g wächst mindestens so schnell wie f $0 < \lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} \leq \infty$ $\Omega(f) := \{g : \mathbb{R} \rightarrow \mathbb{R} \mid \exists c > 0, n_0 \in \mathbb{N} : \forall n > n_0 : g(n) \geq cf(n)\}$
$g = \Theta(f)$	g wächst genauso schnell wie f $0 < \lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} < \infty$ $\Theta(f) = O(f) \cap \Omega(f)$ $c_1 f(n) \leq g(n) \leq c_2 f(n)$
$g = o(f)$	g wächst langsamer als f $\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = 0$ $o(f) := \{g : \mathbb{R} \rightarrow \mathbb{R} \mid \forall c > 0 \exists n_0 \in \mathbb{N} : \forall n > n_0 : g(n) \leq cf(n)\}$
$g = \omega(f)$	g wächst schneller als f $\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = \infty$ $\omega(f) := \{g : \mathbb{R} \rightarrow \mathbb{R} \mid \forall c > 0 \exists n_0 \in \mathbb{N} : \forall n > n_0 : g(n) \geq cf(n)\}$

$1 < \log_{10}(n) < \ln(n) < \log_2(n) < \sqrt{n} < n < n \cdot \ln(n) < (\log n)! < n^2 < e^n < n! < n^n < 2^{2^n}$
Aber $\{\log_{10}(n), \ln(n), \log_2(n)\} \in \Theta(\log n)$

1.1.1 Rechenregeln

Für O, Ω, Θ :

- $cf(n) = O(f(n))$
- $O(f(n)) + O(g(n)) = O(f(n) + g(n)) = O(\max(f(n), g(n)))$
- $O(f(n))O(g(n)) = O(f(n)g(n))$
- $\log n < \sqrt{n} < n < n \log n < n^{3/2} < n^2 < n^3 < 1.1^n < 2^n < n!$

1.1.2 Summenregeln

- $\sum_{i=1}^n c = cn$
- $\sum_{i=1}^n i = \frac{n(n+1)}{2}$
- $\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$
- $\sum_{i=0}^n a^i = \frac{a^{n+1}-1}{a-1}$

1.2. Rekursion

1.2.1 Mastertheorem

Rekursionsgleichung aufstellen und rekursiven Fall betrachten: Seien $a \geq 1, b \geq 1$ Konstanten
 $T(n) = aT(\frac{n}{b}) + f(n)$
1. Fall: Falls $f(n) = O(n^{\log_b(a)-\epsilon})$, $\epsilon > 0$ gilt:
 $T(n) = \Theta(n^{\log_b a})$
2. Fall: Falls $f(n) = \Theta(n^{\log_b a} \log^k n)$, $k \geq 0$ gilt:
 $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$

2. Wichtige Algorithmen

2.1. Sortieralgorithmen

2.1.1 SelectionSort

Sucht das kleinste Element in der Liste, fügt es am Anfang einer neuen Liste ein und löscht es aus der alten Liste. Wiederholt, bis alte Liste leer ist $\rightarrow O(n^2)$

2.1.2 InsertionSort

- Wähle num = input[j]
- Vergleiche alle Indices $j < i$ und schiebe num nach input[j], wenn input[j] < num

Laufzeit	Best case: $O(n)$ falls Input sortiert
	Worst case: $O(n^2)$ falls Input absteigend sortiert
	$O(hn)$ falls ein Element höchstens h Elemente von seiner Zielposition entfernt ist

2.1.3 MergeSort

Liste wird rekursiv halbiert, bis zwei Listen mit je einem Element entstehen. Diese werden jeweils sortiert und mit anderen Teilergebnissen zusammengefügt

Laufzeit	Worst case: $T(n) = 2T(\frac{n}{2}) + O(n) \Rightarrow O(n \log_2 n)$
----------	---

<pre>mergeSort(List* a): if (a->length() <= 1) return a; List* b = a->half(); a = mergeSort(a); b = mergeSort(b); return merge(a,b); half(): Node* ptr = head; for (0 to length/2-2) ptr->next; List* L = new (ptr->next); ptr->next = NULL; return L;</pre>	<pre>merge(a, b): if (a->empty()) delete a; return b; if (b->empty()) delete b; return a; List* h; if a[0] < b[0] h = a; else h = b; int d = h->delFront(); List L = merge(a,b); L->insertFront(d); return L;</pre>
---	--

2.1.4 QuickSort

- Wähle ein Pivotelement A[p]
- Wähle Index i, j = start. Gehe durch die gesamte Liste (überspringe Pivot).
 - Falls A[j] > A[p] $\rightarrow j++$
 - Falls A[i] < A[p] $\rightarrow \text{swap}(i, j), i++, j++$
- Teile Liste in zwei Teile, jeweils links und rechts von A[p] (ohne A[p])

Laufzeit	Worst case: $T(n-1) + O(n) \rightarrow O(n^2)$ falls als Pivotelement immer das größte Element gewählt wird (sehr selten) Average: $T(\frac{n}{2}) + O(n) \rightarrow O(n \log n)$
----------	---

```
quickSort(A, left, right):
if(right <= left) return
pivotIdx = partition(A, left, right)
quickSort(A, left, pivotIdx - 1)
quickSort(A, pivotIdx + 1, right)
```

```
partition(A, left, right):
i = left
pivot = A[r]
for(j = left; j <= right; j++)
    if(A[j] <= pivot)
        swap(A, i, j)
        i++
return i - 1
```

2.2. Selektionsalgorithmen

Man sucht das k -kleinste Element in einer Liste

2.2.1 QuickSelect

Idee: QuickSort, aber rekursiver Aufruf nur in der passenden Hälfte

2.2.2 BFPRT Algorithmus

Findet Median in einer nichtsortierten Liste

Laufzeit | Worst case: $T(\frac{n}{5}) + T(\frac{7n}{10}) + O(n) \rightarrow O(n)$

3. Datenstrukturen

3.1. Sequenz

3.1.1 Sortierte Sequenz

Jedes Element besitzt einen *key*, nach welchem die Elemente sortiert werden (z.B. *std::map*).

3.1.2 Stack

Datenstruktur nach dem *Last In - First Out* Prinzip. Elemente können oben angefügt werden und von oben gelöscht werden.

3.1.3 Queue

Datenstruktur nach dem *First In - First Out* Prinzip. Elemente können am Ende angefügt werden und von vorne gelöscht werden.

3.1.4 (Addressable) Priority Queue

- Einfügen von Elementen mit *key*
- Entfernen von Element mit **minimalem** *key*
- Ansehen des Elements mit **minimalem** *key*
- Für Addressable Priority Queue
 - Insert gibt einen Handle zurück, womit man das jeweilige Element (nicht unbedingt das Minimum) löschen kann
 - decrease-key*: Ändert den Schlüssel von dem Handle dereferenzierten Objekts

Implementiert mithilfe Heap oder

Implementierung mithilfe Array

Speichert Knoten von oben nach unten und links nach rechts (*root, root->left, root->right, root->left->left, ...*)

- Wurzel an $i = 0$
- Knoten an Position i

- Elternknoten an Position $(i - 1)/2$
- linkes Kind an Position $2i + 1$
- rechtes Kind an Position $2i + 2$

4. Graphen

$G = (V, E)$ mit V : Menge der Knoten (n), G : Menge der Kanten (m).

v adjazent zu u (u, v) bzw. $\{u, v\} \in E$
inzident Knoten v und Kante e , wenn $e = \{x, v\}$

4.1. Darstellung von Graphen

4.1.1 Adjazenzmatrix

Speichert vorhandenen Kanten in $A \in \mathbb{R}^{n \times n}$ Matrix. Eine Zeile gehört zu einem Knoten

- $A(i, j) = 1$ (= Gewicht, bei gewichtetem Graph), falls Kante von v_i zu v_j existiert
- $A(i, j) = 0$, sonst

Symmetrisch bei ungerichtetem Graphen. Kanten einfügen löschen einfach, Knoten einfügen/löschen schwierig. Hohen Speicherverbrauch

4.1.2 Adjazenzliste

Speichert für jeden Knoten eine verkettete Liste mit Nachbarn des jeweiligen Knotens; die Knoten selbst werden woanders gespeichert (Array z.B.)

4.1.3 Statischer Graph

Array mit allen Kanten, wobei ein Bereich im Array zu den Verbindungen eines bestimmten Knotens gehört. Array mit allen Knoten, wobei für jeden Knoten der Index seines Bereichs und die Menge an Nachbarn gespeichert wird

5. Bäume

5.1. Binärbaum

Eigenschaften

- Baum
- höchstens zwei ausgehende Kanten pro Knoten

5.2. Heap std::priority_queue

Eigenschaften

- Die Kinder haben immer einen größeren (/gleichen) Schlüssel als der Elternknoten
- Elemente werden *von links nach rechts* eingefügt, ist eine Zeile voll, so geht man wieder nach links und eine Zeile weiter runter
- Verwaltet Zeiger auf das letzte Element
- Suchen $O(n)$, Einfügen/Löschen $O(\log n)$

Einfügen

- Füge Element nach dem letzten Element (am Nachfolger) ein
- Tausche Kind und Elternknoten, bis die Heap-Eigenschaft wieder erfüllt ist

Löschen

1. Ersetze zu löschende Element mit dem letzten Element (x) und lösche das vorherige letzte Element
2. Heap Eigenschaft kann entweder an dem neuen Elternknoten oder an beiden Kindknoten verletzt sein
 - Ist der Elternknoten größer als x , so tausche Kind und Elternknoten, bis die Heap-Eigenschaft wieder erfüllt ist
 - Ist x größer als beide Kinder von x , so tausche x und das kleinere Kind, bis ...

5.3. Binäre Suchbaum

Implementiert eine sortierte Sequenz.

Eigenschaften

1. Binärbaum
2. Alles was links eines Knotens ist ist kleiner und alles rechts eines Knotens ist größer
3. $\emptyset$ Laufzeit der Operationen $O(h)$ mit h als Höhe des Baums (oft $\log n$). Worst case $O(n)$

Operationen

- Löschen**
1. Fall: Element hat keine Kinder oder keinen Nachfolger: Einfach löschen
 2. Fall: Element hat ein Kind: zu löschende Element überbrücken: Elternknoten mit Kind verbinden
 3. Fall: Element hat zwei Kinder: Nachfolger bestimmen, Nachfolger überbrücken, zu löschende Element mit Nachfolger ersetzen

Nachfolger Das Element, welches am wenigsten größer als mein Bezugselement ist. Entweder im rechten Teilbaum oder man geht nach oben und sucht das erste Element, von dem man ein linker Teilbaum ist

5.4. Balancierter Suchbaum `std::set`, `std::map`

Eigenschaften

1. Binärer Suchbaum, wobei die Höhe des Baumes minimal ($\approx O(\log n)$) gehalten wird
2. Alle Operationen garantiert $O(\log n)$

Operationen

Einfügen Fix-Up-Routine muss ab dem Elternknoten des eingefügten Elements aufgerufen werden

Löschen Fix-Up-Routine muss ab dem Elternknoten des gelöschten Elements aufgerufen werden

5.5. AVL-Baum

Eigenschaften

1. Balancierter Suchbaum
2. An jedem Knoten gilt: Höhenunterschied (*Balance*) von linkem und rechtem Teilbaum ist höchstens 1
3. Höhe des Baums immer $O(\log n)$
4. $Balance(\text{Knoten}) = \text{height}(\text{links}) - \text{height}(\text{rechts})$
5. Die *Balance* eines Knotens wird in diesem gespeichert
6. In den Blattknoten werden keine Daten gespeichert (*Dummy*-Knoten, zählen nicht zur Höhe)

Balance eines Knotens wiederherstellen:

```
Balance(v)
if (v->balance == -2)
  if (v->right->balance ∈ {0, -1})
    v = LeftRotate(v)
  else
    v->right = RightRotate(v->right)
    v = LeftRotate(v) // DoubleLeftRotate
else if (v->balance == +2)
  if (v->left->balance ∈ {0, 1})
    v = RightRotate(v)
  else
    v->left = LeftRotate(v->left)
    v = RightRotate(v) // DoubleRightRotate
return v
```

6. Hashing

Universum U Alle möglichen *Keys* (rießig)

Hashtabelle Array $T[0, \dots, n-1]$

Hashfunktion $h : U \rightarrow [0, \dots, n-1]$. Bildet jeden möglichen *Key* auf eine Index in der Hashtabelle ab. Sollte die *Keys* möglichst gleichmäßig verteilen. Viele Kollisionen, da nicht injektiv

6.1. Hash-Tabelle `std::unordered_set`, `std::unordered_map`

Suchen nach Schlüssel, Einfügen, Löschen $\emptyset$ $O(1)$ (Worst case: $O(n)$)

6.2. Hash-Strategien

Direkte Addressierung *Key* wird als direkten Index in den Array der Daten verwendet (Identitätsabbildung)

Perfektes Hashing Man kennt alle möglichen *Keys* und kann somit Kollisionen vermeiden (perfekte Hashfunktion)

6.3. Kollision

Problem: Mehrere verschiedene *Keys* zeigen auf dieselbe Speicherposition

6.3.1 Open Addressing

Alle Objekte werden in der Hashtabelle gespeichert mit Hashfunktion $h(key, 0), \dots, h(key, n-1)$. Ist $h(key, 0)$ belegt, so wird das Element an $h(key, 1)$ eingefügt (ist das belegt, so in $h(key, 2)$ usw.)

Lineares Sondieren $h(k, i) = h(k) + i \mod n$: Element wird an die nächste freie Stelle eingefügt $\rightarrow$ sehr hohe Cache-Effizienz

Quadratisches Sondieren $h(k, i) = h(k) + c_1 i + c_2 i^2 \mod n$

Doppeltes Hashing $h(k, i) = h_1(k) + i h_2(k) \mod n$

6.3.2 Hashing with Chaining

Elemente, welche auf die gleiche Position hashen werden in einer *Linked List* gespeichert und verkettet

6.4. Löschen von Elementen

Problem: Elemente können nicht einfach gelöscht werden.

- **Dummy-Element** wird an die gelöschte Position eingefügt, damit die Probsequenz nicht abgebrochen wird (wird beim Suchen übersprungen)
Problem: Löscht man viele Elemente, so braucht man sehr lange, um ein Element zu suchen